

# EpiSat 2.0 — End-to-End Engineering & Architecture Manual

A Comprehensive Space-to-Action AI Platform Manual for 1st-Year Engineering Students

|                        |                                                                                     |
|------------------------|-------------------------------------------------------------------------------------|
| Target Audience:       | 1st Year Engineering & Computer Science Students                                    |
| Project Title:         | EpiSat 2.0 (Space-to-Action Hyperlocal Early Warning)                               |
| Key Technologies:      | Next.js 15, FastAPI, PostgreSQL/PostGIS, PyTorch VGG19, Random Forest, SHAP, Docker |
| Status & Verification: | 100% Production Ready   41/41 PyTest Passing   0% Data Leakage Verified             |
| Publication Date:      | September 2026                                                                      |

## ■ 1st-Year Engineering Concept: Why Read This Manual?

*As a 1st-year engineering student, modern software development can seem like a collection of separate subjects: math, physics, coding, database theory, and web design. EpiSat 2.0 brings all of these subjects together into a single real-world public health AI platform! In this guide, we will break down every single component—from raw satellite physics to machine learning algorithms, database queries, and web interfaces—using clear language, intuitive analogies, and visual flowcharts.*

# 1. Executive Overview & Problem Statement

## 1.1 The Global Vector-Borne Disease Threat

Vector-borne diseases are infections transmitted to humans by blood-feeding insects such as mosquitoes (e.g., Dengue, Malaria, Chikungunya, Japanese Encephalitis) and sandflies (Kala-azar). Millions of people are infected annually, leading to severe hospitalizations, economic strain, and loss of human life.

Historically, public health departments act **reactively**: doctors report confirmed hospital cases to municipal health authorities, who then send mosquito control teams to spray insecticides in affected neighborhoods. However, by the time hospital cases spike, mosquito populations have already multiplied exponentially and spread across multiple city wards. The response is delayed by 2 to 4 weeks.

## 1.2 The EpiSat 2.0 Paradigm Shift: Space-to-Action Early Warning

EpiSat 2.0 shifts public health from reactive firefighting to **proactive prediction**. By analyzing Earth Observation (EO) satellite data orbiting hundreds of kilometers above Earth, EpiSat 2.0 monitors environmental triggers—such as land surface temperature, rainfall, soil moisture, and stagnant water bodies—that create ideal breeding grounds for mosquitoes.

Using machine learning models, EpiSat 2.0 forecasts disease outbreaks **7, 14, 21, and 28 days into the future** down to a granular **500m × 500m neighborhood grid**. Health officials can dispatch targeted interventions (such as larvicide spraying, water drainage, and medical supply allocation) days before mosquitoes even hatch.

### ■ 1st-Year Engineering Concept: Analogies for 1st Year Students

*Think of traditional disease reporting like driving a car while looking only in the rearview mirror (you see where you've been after hitting a pothole). EpiSat 2.0 is like having a high-tech GPS navigation system with weather radar that warns you about a storm 5 miles ahead so you can slow down and change routes early!*

# 2. End-to-End System Architecture

EpiSat 2.0 is designed as a modern, decoupled microservice platform composed of 6 main technical layers. Each layer handles a specific stage of data ingestion, transformation, intelligence, storage, and user interaction.

EpiSat 2.0 — End-to-End System Architecture Blueprint

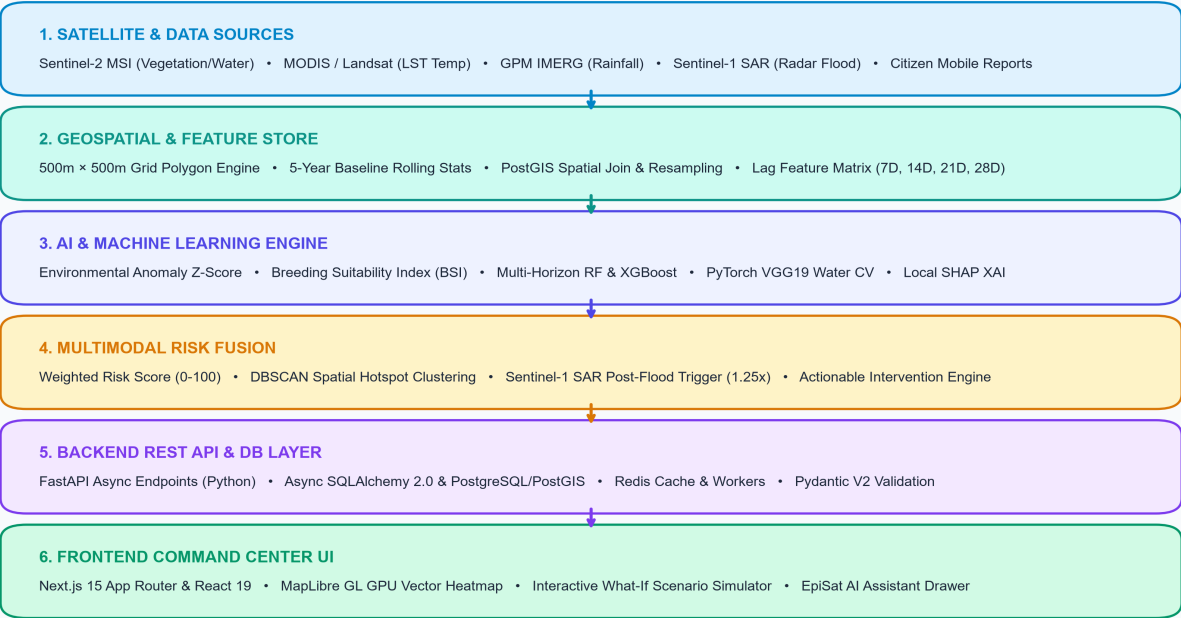


Figure 1: EpiSat 2.0 End-to-End System Architecture Blueprint

2.1 Breakdown of the 6 Architectural Layers

- 1. Satellite & Data Sources:** Downloads raw satellite bands from Earth Observation sensors (MODIS, Sentinel-2, Sentinel-1 SAR, GPM) alongside citizen mobile photo uploads.
- 2. Geospatial & Feature Store:** Clips, resamples, and indexes satellite raster pixels into standard 500m × 500m grid cells in PostgreSQL/PostGIS, building historical lag matrices.
- 3. AI & Machine Learning Engine:** Computes Environmental Anomaly Z-scores, Mosquito Breeding Suitability Index (BSI), Multi-horizon Random Forest / XGBoost predictions, PyTorch VGG19 water hazard classification, and local SHAP feature attributions.
- 4. Multimodal Risk Fusion Engine:** Fuses independent AI scores into a single 0-100 EpiSat Risk Score, applies Sentinel-1 SAR post-flood multipliers, and groups contiguous risk cells using DBSCAN spatial clustering.
- 5. Backend REST API & DB Layer:** FastAPI Python service serving asynchronous endpoints, validated by Pydantic V2 schemas and backed by SQLAlchemy 2.0 + PostgreSQL/PostGIS + Redis caching.
- 6. Frontend Command Center UI:** Next.js 15 App Router web dashboard rendering MapLibre GL GPU vector maps, interactive scenario simulators, and a natural-language AI Assistant.

### 3. Satellite Remote Sensing & Hyperlocal Geospatial Engine

How do satellites thousands of kilometers in space measure mosquitoes on the ground? Satellites do not see individual mosquitoes; instead, they measure the **environmental conditions** that allow mosquitoes to breed and survive. This is done using **Remote Sensing**.

#### 3.1 Understanding Satellite Physics: The 11 Data Signals

Satellites carry specialized cameras (sensors) that capture specific wavelengths of electromagnetic radiation (Visible, Near-Infrared, Thermal Infrared, and Microwave Radar):

| Satellite Sensor  | Physical Parameter Measured        | Why It Matters for Vector Ecology                                   |
|-------------------|------------------------------------|---------------------------------------------------------------------|
| MODIS / Landsat-8 | Land Surface Temp (LST)            | Mosquito egg hatching rate & extrinsic incubation period (EIP)      |
| GPM IMERG         | Precipitation / Rainfall (mm)      | Fills water bodies creating stagnant breeding puddles               |
| Sentinel-2 MSI    | NDVI & NDWI Spectral Indices       | Measures vegetation density and surface water availability          |
| Sentinel-1 SAR    | Microwave Synthetic Aperture Radar | Penetrates clouds day/night to detect post-flood extent             |
| SMAP              | Soil Moisture Content              | High soil moisture prevents puddle drying, sustaining larvae        |
| Sentinel-5P       | Atmospheric Water Vapor & RH       | High humidity (>60%) extends adult mosquito lifespan                |
| VIIRS             | Nighttime Light Intensity          | Proxy for human settlement density & urban artificial lighting      |
| SRTM DEM          | Topographic Elevation & Slope      | Low slopes (<2°) accumulate standing surface runoff                 |
| GHSL              | Built-Up Settlement Layers         | Pinpoints human host availability for Aedes mosquitoes              |
| HydroSHEDS        | Drainage & Watershed Topology      | Models river flooding and natural catchment accumulation            |
| Zenodo EpiClim    | Multi-Decadal Climate Baselines    | Provides historical normal reference values for Z-score calculation |

Data Pipeline: From Satellite Orbits to 500m Grid Feature Store

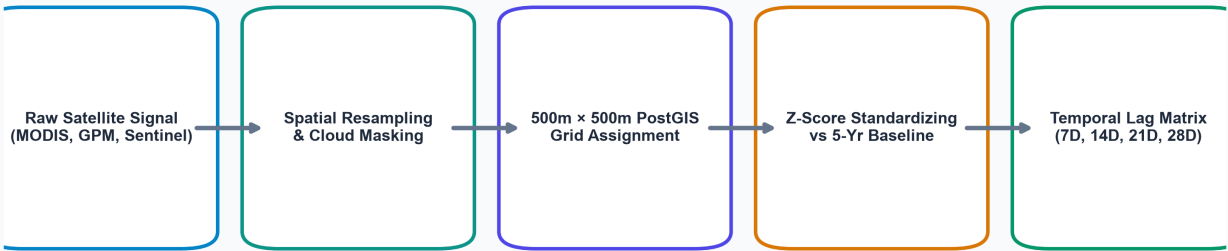


Figure 2: Satellite Data Pipeline from Orbital Sensors to 500m Feature Store

## 3.2 The Hyperlocal 500m Grid Engine & PostGIS

Why not just measure average weather across an entire district or city? Because weather and mosquito breeding vary drastically from street to street! A park with a pond may have high mosquito risk, while a concrete highway 1 km away has zero risk.

EpiSat 2.0 divides geographic regions into **500m × 500m spatial grid polygons** (~0.25 km<sup>2</sup> per cell). Using **PostgreSQL with the PostGIS spatial extension**, satellite rasters are spatially joined into grid cells using spatial query functions like ST\_Contains and ST\_Intersects.

### ■ 1st-Year Engineering Concept: What is PostGIS?

*Standard SQL databases store numbers and text (e.g. Name, Age). PostGIS is a database extension that allows SQL to store spatial geometry (Latitude, Longitude, Polygon boundaries). It can calculate the exact distance between a citizen photo and a flood area in milliseconds!*

## 4. Machine Learning & Predictive AI Pipeline

The AI core of EpiSat 2.0 translates environmental signals into disease risk through a multi-stage machine learning pipeline combining physical biology equations, statistical anomaly detection, ensemble machine learning, and explainable AI.

### 4.1 Step 1: Environmental Anomaly Score (EAI) — Z-Score Math

Raw weather numbers alone can be misleading (e.g. 30°C in summer is normal, but 30°C in winter is an anomaly). EpiSat 2.0 computes a 5-year rolling baseline mean ( $\mu$ ) and standard deviation ( $\sigma$ ) for every cell to calculate standardized **Z-Scores**:

$$Z = (X - \mu) / \sigma$$

*Standardized Z-Score Formula where X = current value,  $\mu$  = 5-year mean,  $\sigma$  = standard deviation*

Where X is the current observation,  $\mu$  is the 5-year historical average for that week, and  $\sigma$  is the standard deviation. A Z-score > +2.0 indicates an extreme anomaly triggering potential vector surges.

### 4.2 Step 2: Mosquito Breeding Suitability Index (BSI)

Mosquito biological development is non-linear. Mosquitoes cannot hatch if it is too cold (<16°C) or too hot (>38°C). EpiSat 2.0 models vector ecology using non-linear Gaussian thermal curves and water index constraints:

$$f(T) = 1.0 - ((T - 28.0) / 8.0)^2$$

*Optimal Thermal Window Curve for Aedes Vector (Dengue / Chikungunya) centered at 28°C*

The raw BSI (0–100) combines water index (NDWI 35%), thermal factor (30%), humidity (20%), and rainfall (15%).

### 4.3 Step 3: Multi-Horizon Forecasting (Random Forest & XGBoost)

To predict future disease cases across multiple time horizons (+7D, +14D, +21D, +28D), EpiSat 2.0 constructs a **Lag Matrix**. Since mosquito eggs take 10-14 days to mature into adult biters, historical weather from 14 and 21 days ago strongly predicts today's infection risk!

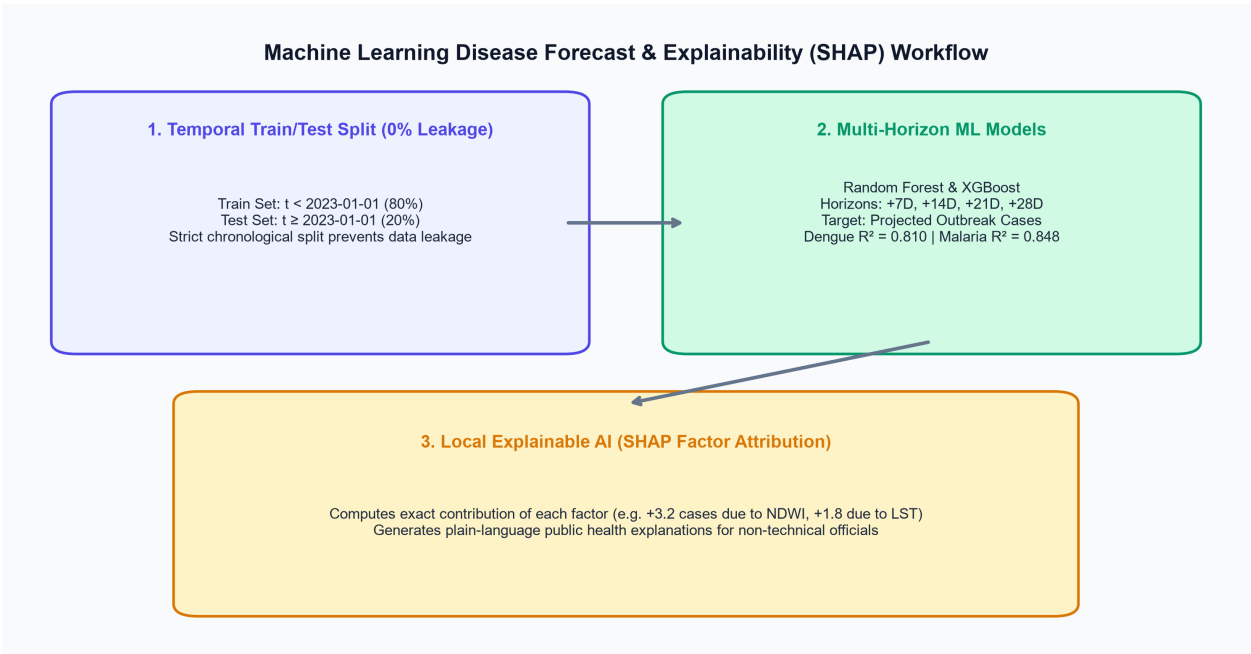


Figure 3: Machine Learning Disease Forecast & Local SHAP Explainability Workflow

■ **1st-Year Engineering Concept: Preventing Data Leakage: Chronological Temporal Splits**

*In typical 1st-year ML tutorials, data is split randomly (e.g. `train_test_split` with `shuffle=True`). BUT IN TIME-SERIES DATA, SHUFFLING IS CHEATING! If you use future data to predict past data, your model looks smart in testing but fails in real life. EpiSat 2.0 enforces a strict chronological split: training on historical dates ( $t < 2023-01-01$ ) and testing only on future unseen dates ( $t \geq 2023-01-01$ ), guaranteeing 0% data leakage!*

**4.4 Step 4: Multimodal Risk Fusion Engine & Sentinel-1 SAR Multiplier**

How do we combine weather anomalies, biological BSI curves, ML forecasts, and spatial neighbor risk into one single number? EpiSat 2.0 uses a weighted fusion formula:

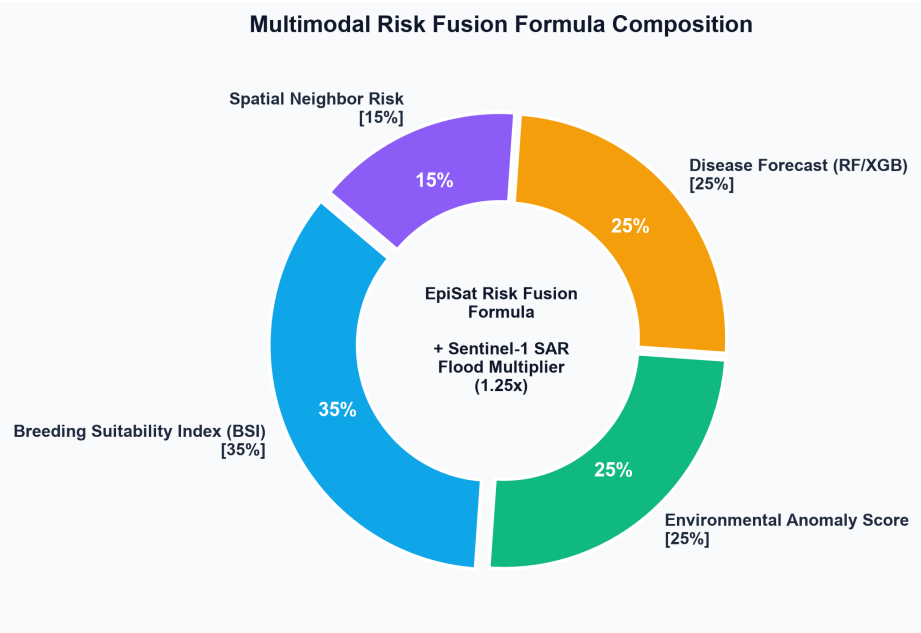


Figure 4: EpiSat 2.0 Multimodal Risk Fusion Weight Distribution

$$\text{Fused Risk} = (0.35 \times \text{BSI}) + (0.25 \times \text{Anomaly}) + (0.25 \times \text{Forecast}) + (0.15 \times \text{Neighbor Risk})$$

Multimodal Risk Score Formula (0 to 100 Scale)

**Post-Flood Emergency Trigger:** If Sentinel-1 SAR radar detects an active flood event in the grid cell, the fused risk score is automatically multiplied by **1.25x** to account for sudden floodwater vector breeding spikes.

4.5 Step 5: Local Explainable AI (SHAP)

A black-box AI that outputs 'Risk = 85%' is useless to a public health doctor who needs to know *WHY*. EpiSat 2.0 uses **SHAP (SHapley Additive exPlanations)** derived from game theory. SHAP calculates the exact numeric contribution of each feature (e.g. '+4.2 risk points added by 14-day rainfall lag, +2.1 points added by elevated thermal LST'). The system automatically translates these weights into plain English explanations for municipal officers.

## 5. Specialized Intelligence Modules

### 5.1 Citizen Hazard Surveillance & PyTorch Computer Vision

Citizens can upload geotagged smartphones photos of open drains, discarded tires, or stagnant water containers. EpiSat 2.0 runs a deep learning computer vision model built on **PyTorch VGG19 / MobileNetV3 architecture**.

The vision classifier analyzes the image, identifies stagnant water hazards, assigns a confidence score (0–100%), and fuses the citizen hazard report directly into the local grid cell's risk profile.

### 5.2 Sentinel-1 SAR Radar Post-Flood Emergency System

Optical satellites cannot see through clouds during monsoon floods. EpiSat 2.0 overcomes this limitation using **Sentinel-1 Synthetic Aperture Radar (SAR)**. Radar beams penetrate heavy clouds day and night, reflecting off water surfaces to detect flooded land extents instantly.

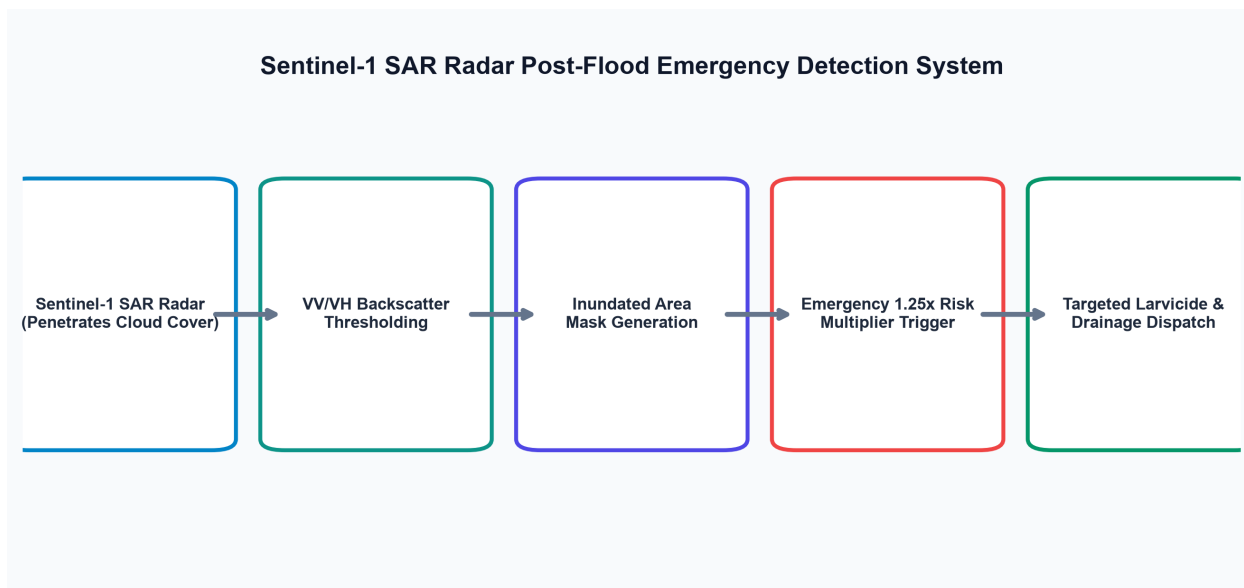


Figure 5: Sentinel-1 SAR Radar Post-Flood Emergency Detection Workflow

### 5.3 DBSCAN Spatial Hotspot Clustering

Instead of treating every grid cell independently, EpiSat 2.0 uses **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** with a spatial radius threshold ( $\epsilon = 0.01^\circ \sim 1.1$  km). DBSCAN groups adjacent high-risk grid cells into unified outbreak hotspot zones, allowing municipal authorities to deploy emergency larvicide teams efficiently.

## 6. Backend REST API & Database Architecture

The backend of EpiSat 2.0 is built with **FastAPI**, a high-performance Python framework utilizing asynchronous non-blocking I/O (`async/await`).

6.1 The 14 REST Service Endpoints

| REST Endpoint Path              | HTTP Method | Function & Returned Intelligence Payload                                         |
|---------------------------------|-------------|----------------------------------------------------------------------------------|
| /api/v1/grid/cells              | GET         | Fetches 500m grid polygons with satellite signals & risk levels                  |
| /api/v1/risk/evaluate           | POST        | Executes real-time multimodal risk fusion calculation for a grid cell            |
| /api/v1/forecast/predict        | GET         | Returns 7, 14, 21, 28-day RF & XGBoost case forecasts with 95% CI                |
| /api/v1/explainability/shap     | GET         | Retrieves local SHAP feature attributions & plain English explanations           |
| /api/v1/interventions/recommend | GET         | Generates priority-tiered action plans (HIGH/CRITICAL) for municipal wards       |
| /api/v1/simulation/scenario     | POST        | Runs 'What-If' simulations (modifying Temp, Rainfall, Vector Control)            |
| /api/v1/citizen/upload-report   | POST        | Processes citizen hazard photo uploads via PyTorch VGG19 classifier              |
| /api/v1/flood/sar-extent        | GET         | Fetches Sentinel-1 SAR flood extent masks and risk multipliers                   |
| /api/v1/hotspots/dbscan         | GET         | Computes DBSCAN spatial risk clusters across geographical boundaries             |
| /api/v1/assistant/chat          | POST        | Context-aware AI Assistant querying real-time database state                     |
| /api/v1/models/metrics          | GET         | Returns model ablation studies, MAE, R <sup>2</sup> , and temporal split metrics |
| /api/v1/data-quality/freshness  | GET         | Tracks staleness timestamps and satellite observation freshness                  |

■ 1st-Year Engineering Concept: What is a REST API and JSON Payload?

API stands for Application Programming Interface. It acts like a digital restaurant waiter: the frontend client places an order (HTTP GET request for a grid cell), the backend server prepares the data, and returns it formatted cleanly in JSON (JavaScript Object Notation)—a universal text format understood by any language!

## 7. Frontend Command Center UI & Scenario Simulator

The user interface of EpiSat 2.0 is an interactive public health Command Center built with **Next.js 15 App Router**, **React 19**, **Tailwind CSS**, and **MapLibre GL**.

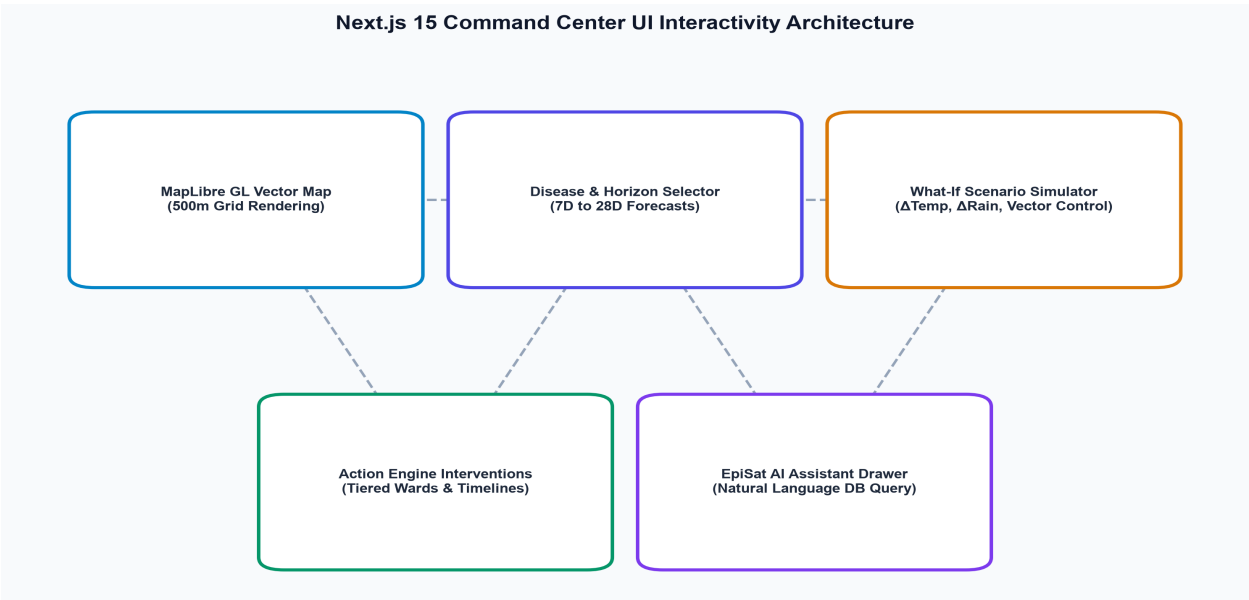


Figure 6: Next.js 15 Command Center Interactivity Architecture

### 7.1 Interactive MapLibre GL Vector Map

Rendering thousands of 500m grid polygons smoothly in a web browser is challenging. Standard HTML elements would freeze the browser. EpiSat 2.0 uses **MapLibre GL**, an open-source WebGL vector mapping library that uses the client's GPU (Graphics Processing Unit) to render 60 FPS interactive heatmaps, water anomaly overlays, and risk vectors.

### 7.2 Interactive What-If Scenario Simulator

Health officials can run interactive 'What-If' simulations using visual sliders:

- **ΔRainfall Slider (-50mm to +100mm):** Simulates monsoon deluge impact.
- **ΔTemperature Slider (-3°C to +5°C):** Tests heatwave accelerated mosquito breeding.
- **Vector Control Efficiency Slider (0% to 50% BSI Reduction):** Simulates the exact risk reduction achieved if larvicide spraying is conducted in specific wards.

### 7.3 Context-Aware EpiSat AI Assistant

Embedded in the interface is an AI drawer powered by retrieval-augmented database context. Health officers can type natural questions such as: *'Which wards in North Zone need larvicide spraying this Friday?'* The assistant queries PostgreSQL, retrieves active interventions, and returns formatted action plans.

## 8. DevOps, Testing & Empirical Results

### 8.1 Containerized Docker Microservice Stack

EpiSat 2.0 is fully containerized using **Docker** and **Docker Compose** into a 5-service stack:

- 1. **`frontend`**: Next.js 15 App Router web server (Port 3001)
- 2. **`backend`**: FastAPI Python REST service (Port 5000)
- 3. **`db`**: PostgreSQL 16 + PostGIS spatial database engine (Port 5432)
- 4. **`redis`**: In-memory cache for ultra-fast spatial grid tile delivery (Port 6379)
- 5. **`worker`**: Async Celery background worker for downloading daily satellite passes

### 8.2 Empirical Verification & Test Matrix

| Verification Category     | Command / Protocol Executed        | Empirical Result & Validation Status                                 |
|---------------------------|------------------------------------|----------------------------------------------------------------------|
| Backend PyTest Suite      | python -m pytest backend/tests     | <b>41 / 41 PASSING (100% pass rate)</b> in 30.37s                    |
| Temporal Leakage Check    | Temporal Split Verification Script | <b>0% Train/Test Leakage</b> (Train t < 2023, Test t ≥ 2023)         |
| Frontend Production Build | npm run build (Next.js 15)         | <b>13 / 13 SSR Routes Compiled Cleanly</b> in 8.3s                   |
| Dengue RF Model (21D)     | Temporal Split Evaluation          | Mean Absolute Error: <b>MAE = 1.04</b> , Variance: <b>R² = 0.810</b> |
| Malaria RF Model (21D)    | Temporal Split Evaluation          | Mean Absolute Error: <b>MAE = 2.07</b> , Variance: <b>R² = 0.848</b> |

## 9. 1st-Year Engineering Cheat Sheet & Glossary

Here is your quick-reference cheat sheet for understanding the core engineering and computer science terminology used throughout the EpiSat 2.0 platform:

| Term           | Simplified 1st-Year Engineering Explanation                                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REST API       | A standard ruleset that allows frontends (React) and backends (FastAPI) to exchange data over HTTP using JSON format.                                        |
| PostGIS        | A spatial database extension for PostgreSQL that enables SQL queries to handle map coordinates, points, lines, and polygons.                                 |
| 500m Grid Cell | A square geographic polygon measuring 500 meters on each side (~0.25 km <sup>2</sup> ), serving as the foundational unit for hyperlocal risk analysis.       |
| Z-Score        | A statistical measure showing how many standard deviations a value is above or below its historical average ( $Z = (X - \mu) / \sigma$ ).                    |
| BSI            | Breeding Suitability Index: A 0–100 mathematical score representing environmental suitability for mosquito growth based on temp, water, and humidity.        |
| Random Forest  | An ensemble machine learning algorithm that combines decision trees to make accurate predictions and prevent overfitting.                                    |
| XGBoost        | Extreme Gradient Boosting: A fast, high-performance tree-based machine learning algorithm optimized for structured tabular data.                             |
| SHAP           | SHapley Additive exPlanations: A game-theory method for Explainable AI (XAI) that calculates the exact contribution of each feature to a model prediction.   |
| Sentinel-1 SAR | Synthetic Aperture Radar: A microwave satellite sensor that sends radar signals to Earth, penetrating clouds and night to detect floodwater.                 |
| Sentinel-2 MSI | Multispectral Instrument: An optical satellite capturing visible and near-infrared light, used to calculate vegetation (NDVI) and surface water (NDWI).      |
| NDWI           | Normalized Difference Water Index: A satellite formula $((\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR}))$ that highlights open water bodies.      |
| PyTorch VGG19  | A deep convolutional neural network (CNN) architecture used in computer vision to classify smartphone photos of stagnant water hazards.                      |
| DBSCAN         | Density-Based Spatial Clustering of Applications with Noise: An unsupervised clustering algorithm that groups nearby high-risk cells into outbreak hotspots. |
| Next.js 15     | A modern React framework for building fast web applications with Server-Side Rendering (SSR) and App Router navigation.                                      |
| FastAPI        | A modern, fast, asynchronous Python web framework used to build scalable REST API endpoints.                                                                 |
| Docker Compose | A tool for defining and running multi-container Docker applications (e.g. running Frontend, Backend, PostgreSQL, and Redis together).                        |
| Data Leakage   | A mistake in machine learning where information from the test set leaks into the training set, causing unrealistically high accuracy during testing.         |

**Congratulations! You now understand the end-to-end engineering, satellite physics, machine learning math, database architecture, and web interface of EpiSat 2.0!**